LoDA-SA: Low-Frequency Distance-Aware Sparse Attention

Anonymous authors

***Abstract*—Long-context sparse attention must retain useful history without spending more on selection than it saves in attention. We present LoDA-SA, a two-level selector that reuses complete low-frequency RoPE coordinate pairs for regional page ranking and pooled token selection, followed by full-dimensional attention on the selected support. Explicit page budgets expose candidate access, while pooled selection removes prescribed regional token quotas. On Qwen3-8B, reported single-run pooled configurations attain 11-task RULER means of 59.02, 47.80 and 46.82 at 32K, 64K and 128K, compared with separate-run dense means of 59.38, 48.54 and 46.43. A region-specific advantage of the scoring rules has not been established, and unmatched execution identities prevent non-inferiority claims. On the primary H20-3e environment, sparse CUDA-Graph implementations report lower steady-state TPOT than archived dense-eager baselines, but excluded preparation and unmatched quality/timing settings preclude a complete-request quality–cost claim.**

***Index Terms*—sparse attention, shared representation, regional retrieval, long-context inference**

I. Introduction

Long-context decoding requires access to useful history while limiting attention cost. Under a finite token budget, a selector must decide which parts of the history can supply candidates and how to rank them. Global ranking does not reserve access for any positional region. Distance-aware allocation makes this choice explicit, without assuming that importance decreases monotonically with distance. Its value depends on both the information retained and the cost of finding it.

Hierarchical selectors shortlist pages before refining individual tokens. AsyncTLS combines inexpensive page summaries with token-level refinement [4]. Mean pooling and sign-aware extrema do not rank pages by their total attention weight. We therefore distinguish page admission from token competition, without assuming that a regional scoring rule establishes better retrieval.

We study LoDA-SA, a two-level selector that combines a shared scoring view with explicit regional page budgets. Its compact Qwen3 configuration uses 32 coordinates comprising 16 complete low-frequency RoPE pairs at both stages. After preparing this view, LoDA-SA ranks near and far pages with averaging and sign-aware extrema, respectively. Absolute near extent n_l and near page budget n_b expose the allocation, with far-page capacity set by subtraction. Nonaligned protection-boundary pages enter outside the page quotas. L2 selects from one pooled token competition, followed by original-dimensional attention over the merged support.

We evaluate the currently selected pooled configuration, $(n_b, n_l) = (32, 4096)$ with regional token quotas disabled. This is the selected operating point, not a claim of a globally optimal configuration. Its reported RULER results and Graph implementation timings are considered separately because their complete execution identities and measurement protocols are not matched.

Our contributions are threefold:

- **Shared low-frequency selection.** We use one compact coordinate view for page ranking and token refinement while retaining original-dimensional Q/K/V for final attention.
- **Explicit regional budgets.** We expose near extent and page allocation, separate page admission from pooled L2 selection, and specify the resulting support and capacity constraints.
- **Evidence separated by scope.** We report the selected pooled configuration's task-level quality and distinguish its execution diagnostics from matched complete-request efficiency.

Task-score proximity alone is insufficient for acceleration. Ordinary L2 still scores full history before masking, so fewer shortlisted pages do not eliminate that work. Whole-step CUDA Graph changes execution overhead, but a quality–latency benefit requires matched quality and complete-request timing, including representation preparation.

II. Related Work

Quest supplies sign-aware page-extremum bounds, SparQ selects key coordinates from queries, and HISA prunes the attention set hierarchically [1], [2], [3]. AsyncTLS explicitly follows Quest for coarse block selection and applies Double Sparsity for fine token selection; its cascade also includes quantization, per-query normalization with GQA probability aggregation and original-KV attention [4]. Earlier Twilight uses a select-then-prune architecture [14]; LoDA-SA studies explicit distance-region page access within this hierarchical-selection paradigm, with complete low-frequency RoPE pairs shared across both stages and unified L2 competition. The relationship is algorithmic; code lineage is unverified. JSSA learns a shared query–key projection and uses gather-free indexed attention [15]; LoDA-SA instead fixes complete low-frequency RoPE pairs and controls page access by distance region.

The selector's own reads also matter. Token-Picker uses conservative probability estimates from partial key-bit chunks to stop further key-chunk and value transfers for pruned tokens [13]; our selector instead ranks candidates with a fixed coordinate subset under page budgets, keeping original-dimensional KV. Our selector keeps both selection stages in bf16: AsyncTLS token keys are INT4, FFD scans 2-bit keys, and official HISA pools block summaries in FP8 [4], [5], [3]; LoDA-SA targets lower selection overhead through compact scoring and structured candidate admission, not numeric compression, and ordinary L2 still scores full history, so narrowing alone establishes no saving. KV compression and placement methods address complementary storage and movement constraints [6], [7], [8]; a smaller scoring view retains original-KV capacity without establishing lower complete-request latency.

III. LoDA-SA: Distance-Aware Selection in a Shared Subspace

LoDA-SA uses a two-level decode-time selector followed by full-dimensional attention over the selected tokens (Fig. 1). The reference configuration prepares a shared 32-coordinate scoring view, ranks near and far pages separately (L1), and selects tokens in a single pool after page masking (L2). The near extent n_l sets the regional boundary; n_b assigns near pages from K_1 ranked-page slots, not token slots. Prefill remains dense. Far gating and L2 candidate gathering are disabled in this reference path; the available quality records bind budget labels but do not fully identify the frozen runtime configuration.

A. Distance regions and candidate budgets

During decoding, t is the zero-based index of the current token, including prefill positions. Let s and w denote the protected sink

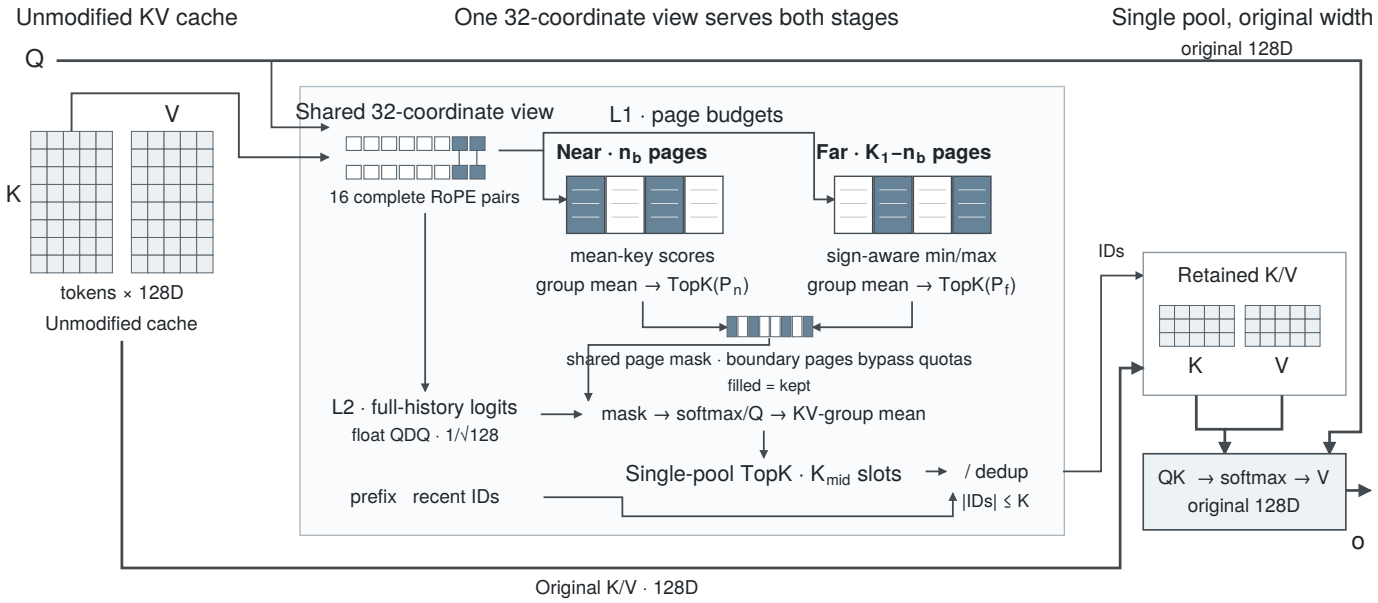


Fig. 1. LoDA-SA reference architecture with boundary bypass enabled and far gating/L2 candidate gathering disabled. Ranked-page budgets are n_b near and $K_1 - n_b$ far; the nominal pooled token count is K_{mid}. A shared coordinate view feeds regional L1 page scoring and L2 token scoring over the full history. Pages intersected by nonaligned protection boundaries join the page mask outside L1 quotas; only protected tokens bypass token selection. Per-query softmax followed by KV-group probability averaging determines one pooled TopK; merging its winners with protected positions yields at most K unique tokens. Selected indices define the support over full-dimensional K/V; physical gathering is backend-dependent. Arrays are schematic.

and recent-window lengths, with protected set $\mathcal{P}_t = \{i : 0 \le i \le t,\ i < s \text{ or } i > t-w\}$ and unprotected middle $\mathcal{M}_t = \{0, \dots, t\} \setminus \mathcal{P}_t$; overlapping protection counts once. History is divided into B-token pages; the final page may be incomplete. For requested near extent n_l, the implementation sets $L = \max(B, n_l)$ and rounds the near/far cut down to a page boundary, so the realized extent can differ from n_l. The near region lies immediately before the protected recent window, and the far region covers earlier middle history. Pages straddling a protection boundary require separate page- and token-eligibility rules; S3 gives the exact integer intervals.

The union of regionally selected pages and protection-boundary bypass pages defines the L2 page mask. L2 still computes logits over the full history; the page mask limits positive-probability positions rather than the scope of logit computation. If positive-probability positions fill the pooled TopK count, masked positions cannot win; otherwise, zero-valued ties may admit positions from rejected pages (S3).

B. Shared low-frequency coordinates

Within each layer and KV head, near and far scoring share one coordinate map across L1 and L2; query heads are scored separately. The Qwen3-8B tail-coordinate configuration selects post-RoPE coordinates $J = \{48, \dots, 63, 112, \dots, 127\}$ from each 128-dimensional query and key. These coordinates form 16 complete low-frequency RoPE pairs under the split-half convention (Supplementary Note S1). With coordinate-selection matrix $E_J \in \mathbb{R}^{128\times 32}$, the scoring vectors are $z_q = E_J^\top q$ and $z_i = E_J^\top k_i$. Their slow positional rotation motivates the choice, but does not guarantee retention of tokens favored by full-dimensional attention. These indices and pairing semantics describe the Qwen3-8B split-half RoPE path, not generic full-width, static-pair, learned-projection or MLA configurations. The reference path uses width-32 scoring tensors; S3 specifies the integer boundaries and support exceptions.

The view reduces each scoring contraction from 128 to 32 coordinates, but this alone establishes no speedup: preparing and maintaining scoring tensors adds work, while original KV remain full-dimensional.

C. Regional page scoring and boundary eligibility

Regional ranked-page quotas and one shared token budget constrain the cascade (Supplementary Fig. S2). For a nonempty page b, the near score is $A_b(z_q) = z_q^\top \bar{z}_b$, where $\bar{z}_b = |b|^{-1} \sum_{i\in b} z_i$. Far pages instead use sign-aware extrema. We use this near-mean/far-extrema assignment as a heuristic; a region-specific advantage of either statistic is not established. Summaries cover full physical pages, including protected positions (S3). Pages straddling nonaligned protection boundaries are excluded from regional L1 ranking and added to its output mask outside the quotas; their unprotected tokens still compete in L2. For a nonempty page b, let $m_{b,j} = \min_{i\in b} z_{i,j}$ and $M_{b,j} = \max_{i\in b} z_{i,j}$. Quest's construction [1] gives

$$U_b(z_q) = \sum_j (z_{q,j}^+ M_{b,j} + z_{q,j}^- m_{b,j}), \tag{1}$$

where $z^+ = \max(z, 0)$ and $z^- = \min(z, 0)$ act coordinatewise. The bound applies to every key in the selected coordinate space: $z_q^\top z_i \le U_b(z_q)$ for $i \in b$. This bound does not bound original-dimensional logits or page attention mass. Extrema from different tokens can make U_b loose and alter page rankings relative to their maximum token scores; mean-key scores have no analogous per-token upper bound.

D. Page quotas and pooled token budget

For total cap $K \ge |\mathcal{P}_t|$, the nominal pooled budget is $K_{\text{mid}} = K - |\mathcal{P}_t|$. After page masking and per-query softmax, probabilities averaged within each KV group determine up to K_{mid} pooled winners,

regardless of region. Pooled TopK can reselect protected positions when the prefix ends inside a page, leaving fewer than K_{mid} distinct unprotected tokens after deduplication (S3). Explicit page quotas expose a dependence rather than remove it: increasing the near quota reduces far-page capacity at fixed K_1. The ranked-page quotas are n_b near and $K_1 - n_b$ far, each clipped to availability; n_l sets the requested near extent.

Pooled L2 does not require both regions to contribute: $n_b = K_1$ leaves no ranked far-page quota, and admitted far pages may yield no final winners. Boundary bypass adds eligible pages independently of ranked quotas; it does not reserve unprotected token slots.

For example, with nonoverlapping protection and sufficient eligible pages, $K = 1024$, $s = 4$, $w = 128$, $K_1 = 128$, $n_b = 32$ and $n_l = 4096$ give 892 nominal pooled slots per KV head and ranked-page quotas of 32 near and 96 far (Supplementary Fig. S2). Boundary bypass pages enter in addition to these quotas.

E. GQA selection and full-dimensional attention

Qwen3-8B assigns four of its 32 query heads to each of its eight KV heads. L1 averages their page scores before regional TopK. L2 computes full-history logits for each query head, applies the shared page mask, then performs per-head softmax. Group-averaged probabilities determine the pooled TopK. L2 uses scale $1/\sqrt{128}$ before softmax even with 32 scoring coordinates. Because softmax precedes group averaging, this rule generally differs from applying softmax to group-averaged logits.

For KV group h, let $\mathcal{S}_{t,h}$ be the deduplicated union of protected positions and pooled winners, with $|\mathcal{S}_{t,h}| \le K$. Each query head g in group h computes full-dimensional attention on this support ($d = 128$):

$$o_{h,g} = \sum_{i \in \mathcal{S}_{t,h}} \frac{e^{q_{h,g}^{\top} k_{i,h}/\sqrt{d}}}{\sum_{j \in \mathcal{S}_{t,h}} e^{q_{h,g}^{\top} k_{j,h}/\sqrt{d}}} v_{i,h}. \tag{2}$$

Simply adding separately normalized regional outputs would not implement this attention; split computation requires denominator-aware recombination, as in SALO [9]. Restricting the support can still change the output relative to dense attention. Physical gathering of selected K/V entries is backend-dependent; short sequences may use a dense masked path.

IV. Experimental Setting

The evaluated strategy is the currently selected pooled configuration: page size 64, $K_1 = 128$, $K = 1024$, near pages/extent $(32, 4096)$, 4/128 sink/recent protection, and no prescribed regional token quota. Its Qwen3-8B reference uses 32 shared tail coordinates, near-mean/far-extrema scoring, dense prefill, sparse decoding and disabled far gating. Table I gives the budget labels. The T6 quality records and Graph execution records agree on these labels but are not one matched experiment: batch size, input, scorer and complete runtime identity differ or remain incompletely bound.

Workloads and comparisons. Quality covers 11 reported RULER [11] tasks at 32K, 64K and 128K, with 100 examples per task and arm. RULER's two QA tasks are not reported and are excluded, not scored as zero. FullKV denotes dense uncapped attention. Separate execution diagnostics cover Qwen3-8B, Llama-3.1-8B-Instruct and Qwen3-32B; their timing prompts are not quality-benchmark samples. Equal nominal budgets do not alone match realized traffic or execution cost.

Metrics. Primary quality is task-score AVG over each fixed benchmark/length set; per-task gains and losses explain AVG without zero-loss requirements. Policy-matched controls require matched models, samples, lengths, scorers, phase policies and aligned sparse-token budgets; subset means are not full-benchmark means. Synchronized TPOT (protocol-specific model-level decode intervals) does not measure preparation-inclusive complete-request latency, so unpaired studies define no quality–latency frontier; AVG differences are descriptive, not non-inferiority evidence.

TABLE I
Reference compact pooled configuration. T6 records report these budget labels; complete runtime identity is not established. K includes unique protected tokens; boundary bypass is outside ranked-page quotas.

Parameter	Setting
Final token cap K	1024
Page size; L1 page cap K_1	64; 128
Near page budget n_b (far)	32 (96)
Near extent n_l	requested 4096; aligned cut
L2 token selection	single pool, free competition
L1/L2 active coordinates	32/32
Prefix / recent protection	4/128

V. Evaluation and Discussion

A. Q1: What quality does sparse retrieval retain?

Current pooled-selection records. Reported configurations at $(n_b, n_l) = (32, 4096)$ attain RULER means of 59.02, 47.80 and 46.82 at 32K, 64K and 128K. Relative to separately recorded dense means of 59.38, 48.54 and 46.43, differences computed before rounding are -0.36, -0.74 and $+0.38$ points. Each length has one reported run and 100 examples for each of the 11 tasks. These observations directly concern pooled token selection; they establish neither non-inferiority nor the isolated effect of replacing fixed quotas. S4 preserves the reported configuration, all task scores and execution-identity limits.

At 128K, the pooled configuration scores 97, 100 and 99 on the three single-needle tasks, while MultiKey-3 remains zero and MultiValue is 3.75. The aggregate therefore does not imply uniform task preservation. Figure 2 exposes all task differences; the complete pooled task-by-length scores and source identities are retained in S4.

B. Q2: What does regional admission guarantee?

Near-mean/far-extrema scoring is a heuristic whose benefit has not been isolated by matched task-level ablation.

Explicit page budgets make regional access inspectable but do not guarantee regional representation in the final support. At fixed K_1, increasing n_b reduces the ranked far-page quota; within the admitted support, one pooled competition may select no far tokens. Nonaligned protection-boundary pages enter outside those quotas. The implementation also clips TopK by interval width rather than positive support, allowing masked zero ties to win when positive support is scarce (S3). The occurrence and quality impact of that behavior in T6 have not been measured. Thus the budget definition and observed task scores do not establish a universal coverage guarantee.

C. Q3: What do the recorded execution costs establish?

Graph-based decoding. On the source-designated primary H20-3e environment, a batch-one implementation reports mean steady-state TPOTs of 24.70, 27.40 and 32.42 ms at nominal 32K, 64K and 128K, against archived dense means of 26.28, 39.67 and 67.04 ms. The sparse arm uses a static cache and whole-step CUDA Graph; the saved dense records use DynamicCache and do not establish graph-enabled execution. Graph TPOT excludes cache filling and capture. The runs therefore establish neither a graph-versus-graph comparison nor preparation-inclusive request savings. A separately

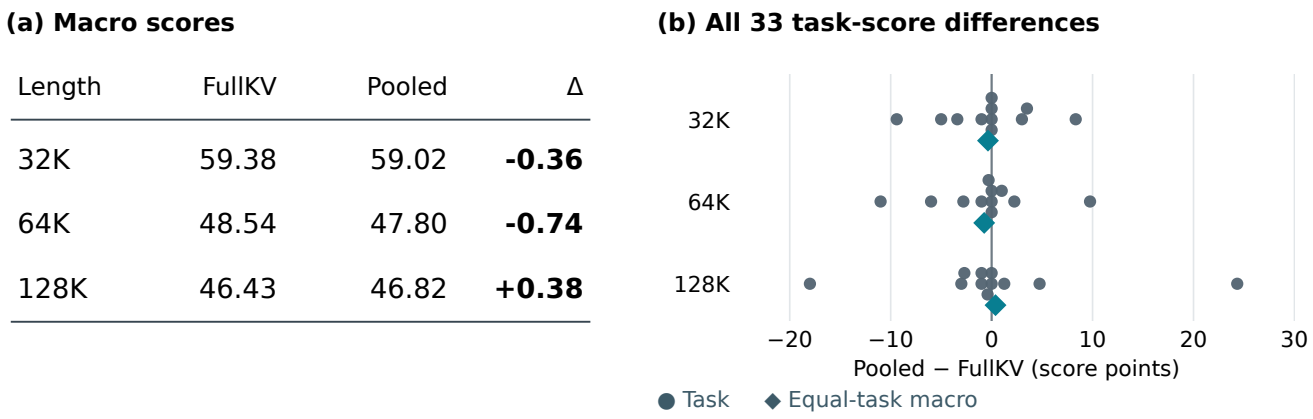


Fig. 2. Current pooled configuration on the 11 reported RULER tasks at three lengths. Aggregate scores and all task-score differences are shown separately; differences are pooled minus dense and are computed before rounding. Each arm reports 100 examples per task. Dense anchors are separate runs with incomplete input/scorer/execution matching. These are descriptive point estimates, not paired uncertainty or non-inferiority evidence.

TABLE II
Steady-state TPOT across models (ms; lower is better). Dense uses eager execution; sparse uses CUDA Graph. Ratios are dense/sparse, not isolated algorithmic speedups. Actual prompt lengths differ from nominal labels. Prefill, cache filling and Graph capture are excluded.

Nominal	Input tokens	Dense eager	Sparse Graph	Ratio
Llama-3.1-8B-Instruct				
32K	32732	21.43	21.12	1.01
64K	65472	33.10	23.47	1.41
128K	130952	56.86	29.00	1.96
Qwen3-32B				
32K	33729	55.50	54.70	1.01
64K	67459	80.21	59.93	1.34
128K[†]	134923	129.10	70.11	1.84

[†]Expandable-segment allocator in both arms; sparse uses four separate processes with one measured run and no discarded warmup each. Dense retains two processes with three measured runs after warmup. Other rows use two processes/arm, with six measured dense runs and six/four sparse runs for Llama/Qwen, respectively. No paired confidence intervals are reported.

reported local-GPU1 replica has higher sparse TPOT at 32K (47.61 versus 39.34 ms); its source attributes interference to a co-tenant, without independent interference logs verified here. The primary environment is held fixed across lengths, rather than selecting the faster environment per row. Their parameter labels overlap the pooled quality snapshots, but full execution identity and batch size are not matched (S4).

Across model families. Table II extends the steady-state comparison to Llama-3.1-8B-Instruct and Qwen3-32B on H20-3e, with batch one and 128 generated tokens. Dense-eager and sparse-Graph means differ little at nominal 32K; their ratio grows to 1.96 for Llama at 128K and 1.34 for Qwen3-32B at 64K. This is a comparison of implementations, not an isolated effect of compact scoring. Qwen3-32B also completes 128K decoding in a separately configured run, with a ratio of 1.84; this row is not part of the same-protocol length sweep. These records extend execution evidence across families and model sizes, but provide no matched task-quality or complete-request comparison (S4).

Limits of the evidence. The quality and execution records describe the selected strategy under their respective protocols. They neither isolate the effects of coordinate width, regional scoring or pooled allocation nor establish a matched quality–cost frontier. In particular, Graph/eager token agreement on timing prompts does not establish quality equivalence to dense attention. Ordinary L2 still scores full history, and representation preparation, maintenance and cache setup remain necessary costs. Far gating is disabled; no claim is made for a new far-skipping or KV-release implementation.

VI. Conclusion

LoDA-SA makes candidate admission and final token competition explicit within a shared low-frequency page-to-token selector. The currently selected pooled configuration provides direct descriptive RULER quality evidence, including heterogeneous task gains and losses. Regional scorer specialization remains unestablished. On the primary H20-3e environment, its Graph implementations record lower steady-state TPOT than dense-eager baselines under different execution regimes. Matched quality and preparation-inclusive complete-request execution remain necessary to establish a quality–cost benefit.

Content Generated by AI. Codex/OpenAI models assisted drafting, revision, translation, figures and code checks; model versions not exposed.

References

[1] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference," in *ICML*, 2024. https://arxiv.org/abs/2406.10774.

[2] L. Ribar, I. Chelombiev, L. Hudlass-Galley, C. Blake, C. Luschi, and D. Orr, "SparQ Attention: Bandwidth-Efficient LLM Inference," in *ICML*, 2024, pp. 42558–42583. https://proceedings.mlr.press/v235/ribar24a.html.

[3] Y. Xu *et al.*, "HISA: Efficient Hierarchical Indexing for Fine-Grained Sparse Attention," in *COLM*, 2026. https://arxiv.org/abs/2603.28458v4.

[4] Hu *et al.*, "AsyncTLS: Efficient Generative LLM Inference with Asynchronous Two-level Sparse Attention," arXiv:2604.07815v1, Apr. 9, 2026. https://arxiv.org/abs/2604.07815v1.

[5] Z. Liu, Z. Ning, R. Li, X. Liu, Y. Song, M. Zhang, Z. He, and X. Qiu, "Faster Than Flash: Exploiting Attention Sparsity for Efficient Long-Context Decoding," in *ICML*, 2026, pp. 77913–77928. https://proceedings.mlr.press/v306/liu26da.html.

[6] H. Sun *et al.*, "ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference," in *ICML*, 2025, pp. 57355–57373. https://proceedings.mlr.press/v267/sun25b.html.

[7] Z. Wang et al., "MILLION: MasterIng Long-Context LLM Inference Via Outlier-Immunized KV Product QuaNtization," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7, doi: 10.1109/DAC63849.2025.11132862.

[8] G. Liu, C. Li, J. Zhao, C. Zhang, and M. Guo, "ClusterKV: Manipulating LLM KV Cache in Semantic Space for Recallable Compression," in *DAC*, 2025. https://doi.org/10.1109/DAC63849.2025.11132479.

[9] G. Shen, J. Zhao, Q. Chen, J. Leng, C. Li, and M. Guo, "SALO: An Efficient Spatial Accelerator Enabling Hybrid Sparse Attention Mechanisms for Long Sequences," in *DAC*, 2022, pp. 571–576. https://doi.org/10.1145/3489517.3530504.

[10] Y. Bai *et al.*, "LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding," in *ACL*, 2024, pp. 3119–3137. https://aclanthology.org/2024.acl-long.172/.

[11] C.-P. Hsieh *et al.*, "RULER: What's the Real Context Size of Your Long-Context Language Models?," in *COLM*, 2024. https://arxiv.org/abs/2404.06654.

[12] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in *ICLR*, 2024. https://arxiv.org/abs/2309.17453v4.

[13] J. Park, M. Kang, Y. Han, Y.-G. Kim, J. Shin, and L.-S. Kim, "Token-Picker: Accelerating Attention in Text Generation with Minimized Memory Transfer via Probability Estimation," in *DAC*, 2024, doi: 10.1145/3649329.3655953.

[14] C. Lin *et al.*, "Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning," arXiv:2502.02770v1, Feb. 4, 2025. https://arxiv.org/abs/2502.02770v1.

[15] J. Song, Y. Kim, and J.-J. Kim, "Joint Subspace Sparse Attention for Efficient Long-Context Decoding," arXiv:2610.19026v1, Oct. 13, 2026. https://arxiv.org/abs/2610.19026v1.